

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

Redovne studije

Smjer : “Inženjering Informacionih Tehnologija“

Predmet:

Konkurentno računarstvo – paralelno programiranje

**UPOREDNA ANALIZA SEKVENCIJALNOG I
PARALELNOG PROGRAMIRANJA U PYTHON
OKRUŽENJU
(Seminarski rad)**

**Predmetni nastavnik
Prof.dr Negovan Stamenković**

**Student: Pavlović Ivan
Br. Indeksa : 92-20/RITP-S**

Banja Luka, jun 2026

SADRŽAJ

UVOD	1
1. PYTHON KAO PROGRAMSKO OKRUŽENJE.....	2
1.1. Istorijski razvoj programskog jezika Python	2
1.2. Primjena Pythona u industriji i nauci.....	2
1.3. Ograničenja sekvencijalnog programiranja	2
1.4. Višejezgreni procesori i potreba za paralelnim programiranjem	3
2. PARALELIZAM U PROGRAMIRANJU	4
2.1. Osnovni pojmovi paralelizma	4
2.1.1. Procesi.....	4
2.1.2. Niti	4
2.1.3. Konkurentnost.....	4
2.1.4. Paralelizam.....	4
2.2. Biblioteke i alati za paralelno programiranje u Pythonu	5
2.2.1. Threading	5
2.2.2. Multiprocessing.....	5
2.2.3. Concurrent.futures	5
2.3. Prednosti paralelnog programiranja	6
2.4. Kada koristiti paralelizam	6
2.5. Ograničenja i izazovi paralelnog programiranja.....	6
3. PRAKTIČNI PRIMJERI I POREĐENJE.....	7
3.1. Primjer 1: Sabiranje elemenata velike liste.....	7
3.2. Primjer 2: Izračunavanje kvadrata brojeva	8
3.3. Primjer 3: Provjera prostih brojeva.....	9
3.4. Primjer 4: Pretraga elementa u velikoj listi.....	11
3.5. Primjer 5: Sortiranje više nezavisnih lista	13
3.6. Primjer 6: Brojanje riječi u tekstovima	14
3.7. Primjer 7: Rad sa više tekstualnih fajlova.....	16
3.8. Primjer 8: Simulacija mrežnih zahtjeva	19
3.9. Primjer 9: Obrada slike predstavljene matricom piksela	20
3.10. Primjer 10: Monte Carlo simulacija za približnu vrijednost broja pi	21
3.11. Primjer 11: Množenje matrica	24
3.12. Primjer 12: Izračunavanje faktoriijela.....	26
3.13. Primjer 13: Filtriranje velikog skupa podataka.....	27

3.14. Primjer 14: Generisanje hash vrijednosti	30
3.15. Primjer 15: Simulacija više nezavisnih zadataka.....	31
ZAKLJUČAK.....	33
POPIS SLIKA.....	34

UVOD

Razvoj savremenih računarskih sistema usko je povezan sa potrebom za sve bržom obradom podataka. U ranijim fazama razvoja računarstva, programi su se uglavnom izvršavali sekvencijalno, što znači da je procesor izvršavao jednu instrukciju za drugom. Takav način programiranja je jednostavan za razumijevanje, lakši za testiranje i pogodan za veliki broj svakodnevnih zadataka. Međutim, sa rastom količine podataka i složenosti problema, sekvencijalno programiranje sve češće postaje ograničavajući faktor.

Savremeni računari najčešće imaju više procesorskih jezgara. To znači da hardver pruža mogućnost da se više zadataka izvršava istovremeno. Da bi softver mogao iskoristiti takve mogućnosti, potrebno je primijeniti tehnike paralelnog programiranja. Paralelno programiranje omogućava da se veći problem podijeli na više manjih zadataka koji se mogu izvršavati istovremeno. Na taj način se može smanjiti vrijeme izvršavanja programa i bolje iskoristiti raspoloživi hardverski resursi.

U ovom radu obrađuje se uporedna analiza sekvencijalnog i paralelnog programiranja u Python okruženju. Python je izabran jer je jednostavan za čitanje, široko primjenjiv i posjeduje više biblioteka koje omogućavaju konkurentno i paralelno izvršavanje programa. Zvanična dokumentacija opisuje Python kao programski jezik koji je jednostavan za učenje, ali istovremeno dovoljno moćan za ozbiljan razvoj softvera

Cilj rada je da se kroz teorijski dio objasne osnovni pojmovi paralelizma, a zatim kroz praktične primjere prikaže razlika između sekvencijalnog i paralelnog pristupa. Posebna pažnja posvećena je mjerenju vremena izvršavanja programa i analizi situacija u kojima paralelizam donosi ubrzanje, ali i situacija u kojima paralelna verzija može biti jednaka ili čak sporija od sekvencijalne.

1. PYTHON KAO PROGRAMSKO OKRUŽENJE

1.1. Istorijski razvoj programskog jezika Python

Python je programski jezik visokog nivoa koji je kreirao Guido van Rossum krajem osamdesetih i početkom devedesetih godina dvadesetog vijeka. Prema zvaničnoj Python dokumentaciji, Python je nastao početkom devedesetih godina u institutu CWI u Holandiji kao nasljednik jezika ABC.

Prva javna verzija Pythona objavljena je 1991. godine. Već u tim ranim verzijama Python je imao osobine koje su ga razlikovale od mnogih drugih jezika, a to su jednostavna sintaksa, čitljiv kod, podrška za module i rad sa osnovnim strukturama podataka. Tokom vremena Python se razvijao kroz verzije Python 1, Python 2 i Python 3. Danas je Python 3 standardna verzija jezika i koristi se u obrazovanju, industriji, nauci i istraživanju.

Jedna od najvažnijih osobina Pythona jeste njegova čitljivost. Kod napisan u Pythonu često izgleda jednostavnije nego kod u jezicima kao što su C++ ili Java. Zbog toga je pogodan za početnike, ali i za profesionalne programere koji žele brzo razvijati prototipove, skripte, alate i kompleksnije aplikacije. Python takođe ima veliku standardnu biblioteku i veliki broj dodatnih paketa, što značajno proširuje njegovu upotrebu.

1.2. Primjena Pythona u industriji i nauci

Python se danas koristi u velikom broju oblasti. U industriji se često koristi za automatizaciju, web razvoj, analizu podataka, testiranje softvera i razvoj skripti koje olakšavaju svakodnevni rad. Njegova popularnost dolazi iz činjenice da omogućava brz razvoj i jednostavno povezivanje sa drugim tehnologijama.

U nauci Python ima posebno značajnu ulogu. Biblioteke kao što su NumPy, pandas, SciPy i matplotlib omogućavaju rad sa velikim skupovima podataka, matematičke proračune, statističku analizu i vizuelizaciju rezultata. U oblasti vještačke inteligencije i mašinskog učenja Python se koristi zajedno sa bibliotekama kao što su TensorFlow, PyTorch i scikit-learn.

Python je pogodan i za eksperimentalno programiranje. Kada istraživač želi brzo provjeriti neku ideju, Python omogućava da se algoritam napiše u relativno kratkom vremenu. Upravo ova kombinacija jednostavnosti i velikog broja biblioteka čini Python jednim od najčešće korišćenih jezika u savremenom softverskom razvoju.

1.3. Ograničenja sekvencijalnog programiranja

Sekvencijalno programiranje podrazumijeva da se instrukcije izvršavaju redom, jedna za drugom. Takav pristup je prirodan i jednostavan za razumijevanje. Na primjer, ako program treba da obradi hiljadu brojeva, sekvencijalna verzija će uzeti prvi broj, obraditi ga, zatim drugi, zatim treći i tako dalje.

Problem se javlja kada se obrađuju veliki skupovi podataka ili kada jedan zadatak može trajati veoma dugo. Ako računar ima više procesorskih jezgara, sekvencijalni program obično koristi samo jedno jezgro za glavni tok izvršavanja. Ostala jezgra tada ostaju djelimično neiskorišćena. To znači da hardver ima mogućnost za brži rad, ali program nije napisan tako da tu mogućnost iskoristi.

Sekvencijalni pristup je posebno ograničen kod problema koji se prirodno mogu podijeliti na nezavisne dijelove. Primjeri su obrada više slika, analiza više fajlova, provjera velikog broja brojeva, simulacije i matematička izračunavanja. U takvim slučajevima paralelna obrada može značajno smanjiti vrijeme izvršavanja.

1.4. Višejezgreni procesori i potreba za paralelnim programiranjem

Razvoj procesora se dugo zasnivao na povećanju radne frekvencije. Međutim, zbog fizičkih ograničenja, potrošnje energije i zagrijavanja, proizvođači procesora počeli su razvijati procesore sa više jezgara. Višejezgreni procesor omogućava da se više zadataka izvršava istovremeno.

Da bi program mogao iskoristiti više jezgara, potrebno ga je organizovati tako da se određeni dijelovi izvršavaju paralelno. To se postiže korišćenjem niti, procesa ili viših programskih biblioteka koje programeru olakšavaju rad sa paralelnim zadacima.

U Pythonu je posebno važno razumjeti razliku između niti i procesa. Zbog postojanja GIL mehanizma u standardnoj CPython implementaciji, niti nisu uvijek pogodne za CPU-intenzivne zadatke. Zvanična dokumentacija objašnjava da GIL dozvoljava da samo nit koja posjeduje zaključavanje izvršava Python objekte u određenom trenutku. Zbog toga se za teške računske zadatke često koristi modul multiprocessing, koji pokreće odvojene procese i na taj način može iskoristiti više procesorskih jezgara.

2. PARALELIZAM U PROGRAMIRANJU

2.1. Osnovni pojmovi paralelizma

Paralelizam u programiranju predstavlja način organizacije programa u kojem se više dijelova programa može izvršavati istovremeno. Da bi se paralelizam pravilno razumio, potrebno je razlikovati pojmove proces, nit, konkurentnost i paralelizam. Ovi pojmovi se često koriste zajedno, ali ne znače isto.

2.1.1. Proces

Proces je nezavisna instanca programa koja se izvršava u operativnom sistemu. Svaki proces ima sopstveni memorijski prostor i resurse. Zbog toga su procesi međusobno izolovaniji od niti. Ako jedan proces ima grešku, ta greška ne mora direktno uticati na memoriju drugog procesa.

Prednost procesa je što se mogu izvršavati na različitim procesorskim jezgrima. U Pythonu se zbog toga procesi često koriste za CPU-intenzivne zadatke. Nedostatak procesa je veći trošak njihovog kreiranja i komunikacije. Kada je potrebno poslati podatke iz glavnog programa u dodatni proces, ti podaci se obično moraju serijalizovati i prenijeti, što može usporiti program.

2.1.2. Niti

Nit je manja jedinica izvršavanja unutar jednog procesa. Više niti može dijeliti isti memorijski prostor. Zbog toga je komunikacija između niti često brža nego komunikacija između procesa. Međutim, dijeljenje memorije uvodi i rizik. Ako dvije niti istovremeno mijenjaju isti podatak, može doći do grešaka koje se teško otkrivaju.

U Pythonu se niti često koriste za I/O zadatke, kao što su rad sa fajlovima, mrežni zahtjevi ili čekanje odgovora od korisnika. Kod takvih zadataka program dosta vremena provodi čekajući, pa niti mogu poboljšati odziv i ukupno vrijeme izvršavanja.

2.1.3. Konkurentnost

Konkurentnost znači da program može upravljati sa više zadataka u istom vremenskom periodu. Ti zadaci ne moraju nužno biti izvršeni baš u istom trenutku. Operativni sistem može brzo prebacivati izvršavanje sa jednog zadatka na drugi, pa korisnik ima utisak da se zadaci odvijaju istovremeno. Konkurentnost je korisna kod aplikacija koje moraju ostati odzivne. Na primjer, grafička aplikacija može u jednoj niti obrađivati korisnički interfejs, a u drugoj niti učitavati podatke sa diska.

2.1.4. Paralelizam

Paralelizam znači da se više zadataka zaista izvršava istovremeno. To je moguće kada računar ima više procesorskih jezgara ili kada se koriste posebne procesorske jedinice kao što su grafički procesori.

Paralelizam je najkorisniji kada se problem može podijeliti na nezavisne cjeline. Ako ti dijelovi ne zavise jedni od drugih, mogu se izvršavati istovremeno. Na primjer, ako treba obraditi deset slika, svaka slika se može poslati posebnom procesu.

2.2. Biblioteke i alati za paralelno programiranje u Pythonu

Python standardna biblioteka sadrži više alata za konkurentno i paralelno izvršavanje. Zvanična dokumentacija navodi da izbor odgovarajućeg alata zavisi od vrste zadatka, odnosno od toga da li je zadatak CPU-bound ili I/O-bound.

2.2.1. Threading

Modul threading omogućava rad sa nitima. Koristi se kada program treba da izvršava više zadataka koji uglavnom čekaju na ulazno-izlazne operacije. Primjeri su čitanje fajlova, komunikacija preko mreže ili čekanje odgovora od servera.

Prednost modula threading je u tome što je relativno jednostavan za korišćenje. Niti se pokreću unutar istog procesa i mogu pristupati istim promjenljivim. Međutim, to istovremeno znači da programer mora voditi računa o sinhronizaciji, zaključavanju i mogućim konfliktima pri pristupu podacima.

2.2.2. Multiprocessing

Modul multiprocessing omogućava pokretanje više procesa. Zvanična Python dokumentacija navodi da ovaj modul koristi podprocese i time zaobilazi Global Interpreter Lock, što omogućava bolje korišćenje više procesora.

Ovaj modul je posebno pogodan za CPU-intenzivne zadatke. To su zadaci u kojima se najviše vremena troši na računanje, a ne na čekanje. Primjeri su matematičke simulacije, provjera prostih brojeva, obrada slika, množenje matrica i analiza velikih skupova podataka.

Nedostatak multiprocessing pristupa jeste veća potrošnja memorije. Svaki proces ima svoj memorijski prostor, pa se veliki podaci mogu kopirati više puta. Takođe, pokretanje procesa ima određeni vremenski trošak, pa se ne isplati za veoma male zadatke.

2.2.3. Concurrent.futures

Modul concurrent.futures pruža viši nivo apstrakcije za rad sa nitima i procesima. On omogućava da se zadaci pokreću asinhrono pomoću klasa ThreadPoolExecutor i ProcessPoolExecutor. Zvanična dokumentacija navodi da ovaj modul pruža interfejs visokog nivoa za asinhrono izvršavanje pozivljivih objekata.

Prednost ovog modula je jednostavnost. Programer može koristiti gotovo isti stil koda za rad sa nitima i procesima. Ako se radi o I/O zadacima, obično se koristi ThreadPoolExecutor. Ako se radi o teškim računskim zadacima, koristi se ProcessPoolExecutor.

2.3. Prednosti paralelnog programiranja

Paralelno programiranje može donijeti značajne prednosti. Najvažnija prednost je smanjenje vremena izvršavanja programa. Ako se zadatak može podijeliti na četiri nezavisna dijela i računar ima četiri jezgra, moguće je da se ti dijelovi izvrše znatno brže nego sekvencijalno.

Druga prednost je bolje iskorišćenje resursa računara. Savremeni računari često imaju više jezgara, ali sekvencijalni programi ih ne koriste u potpunosti. Paralelni programi mogu bolje rasporediti posao na raspoloživi hardver.

Treća prednost je skalabilnost. Program koji je dobro paralelizovan može se lakše prilagoditi većem broju podataka i jačem hardveru. Na primjer, ako se poveća broj procesorskih jezgara, paralelni program može u određenim slučajevima dodatno ubrzati izvršavanje.

2.4. Kada koristiti paralelizam

Paralelizam treba koristiti onda kada postoji stvarna potreba i kada priroda problema to omogućava. Najpogodniji su zadaci koji se mogu podijeliti na nezavisne dijelove. Ako jedan dio ne mora čekati rezultat drugog dijela, problem je dobar kandidat za paralelizaciju.

Paralelizam je koristan kod obrade velikih skupova podataka, obrade slika, simulacija, rada sa više fajlova, mrežnih zahtjeva i numeričkih proračuna. Na primjer, ako program mora obraditi više hiljada fajlova, svaki fajl se može obrađivati nezavisno. Ako program mora provjeriti veliki broj brojeva, provjera se može rasporediti na više procesa.

Ipak, paralelizam nije univerzalno rješenje. Ako je zadatak mali ili veoma jednostavan, trošak kreiranja niti ili procesa može biti veći od dobiti. Zbog toga je mjerenje vremena izvršavanja veoma važno.

2.5. Ograničenja i izazovi paralelnog programiranja

Paralelno programiranje uvodi dodatnu složenost. Program više ne izvršava samo jedan tok instrukcija, nego više tokova koji mogu međusobno zavisiti. To znači da se mogu pojaviti problemi kao što su race condition, deadlock i nekonzistentni podaci.

Race condition nastaje kada više niti ili procesa pristupa istom podatku, a krajnji rezultat zavisi od redoslijeda izvršavanja. Deadlock nastaje kada dva ili više zadataka čekaju jedni druge i nijedan ne može nastaviti izvršavanje.

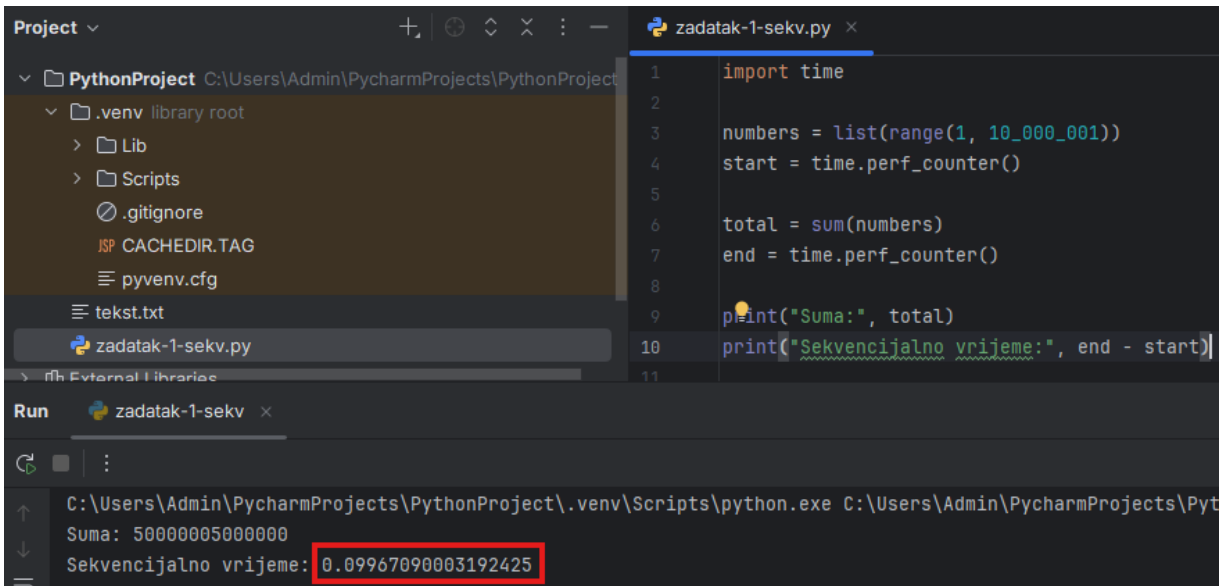
Još jedan izazov je overhead, odnosno dodatni trošak. Kreiranje procesa, komunikacija između procesa, kopiranje podataka i sinhronizacija mogu usporiti program. Zato paralelna verzija nije automatski bolja od sekvencijalne. Uspjeh zavisi od veličine problema, složenosti operacije i količine podataka koji se prenose između dijelova programa.

3. PRAKTIČNI PRIMJERI I POREĐENJE

3.1. Primjer 1: Sabiranje elemenata velike liste

Opis problema

Potrebno je izračunati sumu svih elemenata velike liste. Sekvencijalna verzija koristi ugrađenu funkciju `sum`, dok paralelna verzija dijeli listu na više dijelova i svaki dio sabira u posebnom procesu.

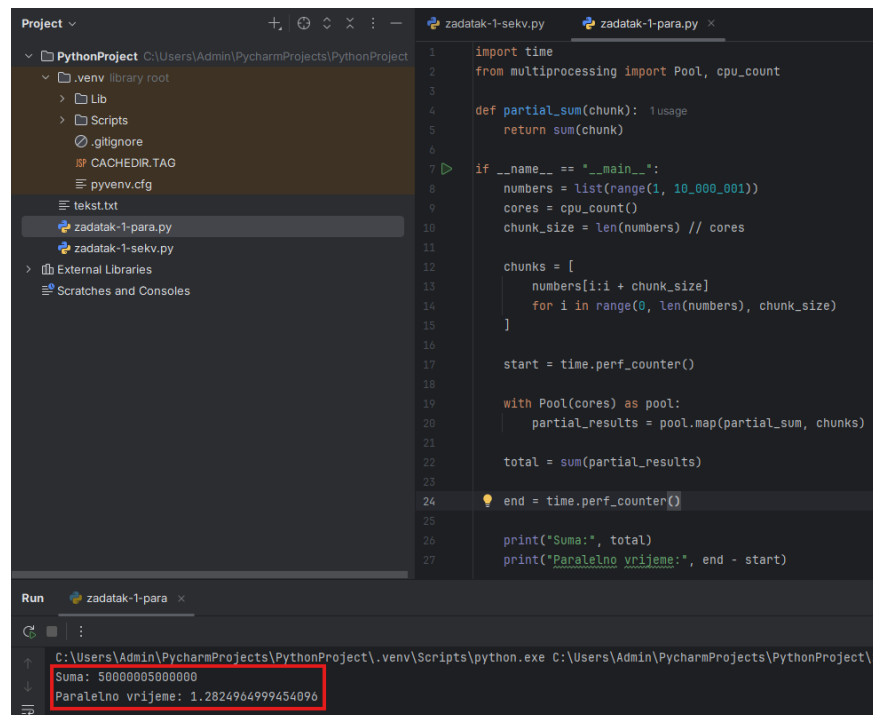


```
Project PythonProject C:\Users\Admin\PycharmProjects\PythonProject
  .venv library root
    Lib
    Scripts
    .gitignore
    CACHEDIR.TAG
    pyvenv.cfg
  tekst.txt
  zadatak-1-sekv.py
  External Libraries

Run zadatak-1-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Pyt
Suma: 50000005000000
Sekvencijalno vrijeme: 0.09967090003192425
```

Slika 1 - Sekvencijalna verzija sabiranja elemenata velike liste



```
Project PythonProject C:\Users\Admin\PycharmProjects\PythonProject
  .venv library root
    Lib
    Scripts
    .gitignore
    CACHEDIR.TAG
    pyvenv.cfg
  tekst.txt
  zadatak-1-para.py
  zadatak-1-sekv.py
  External Libraries
  Scratches and Consoles

Run zadatak-1-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\z
Suma: 50000005000000
Paralelna vrijeme: 1.2824964999454096
```

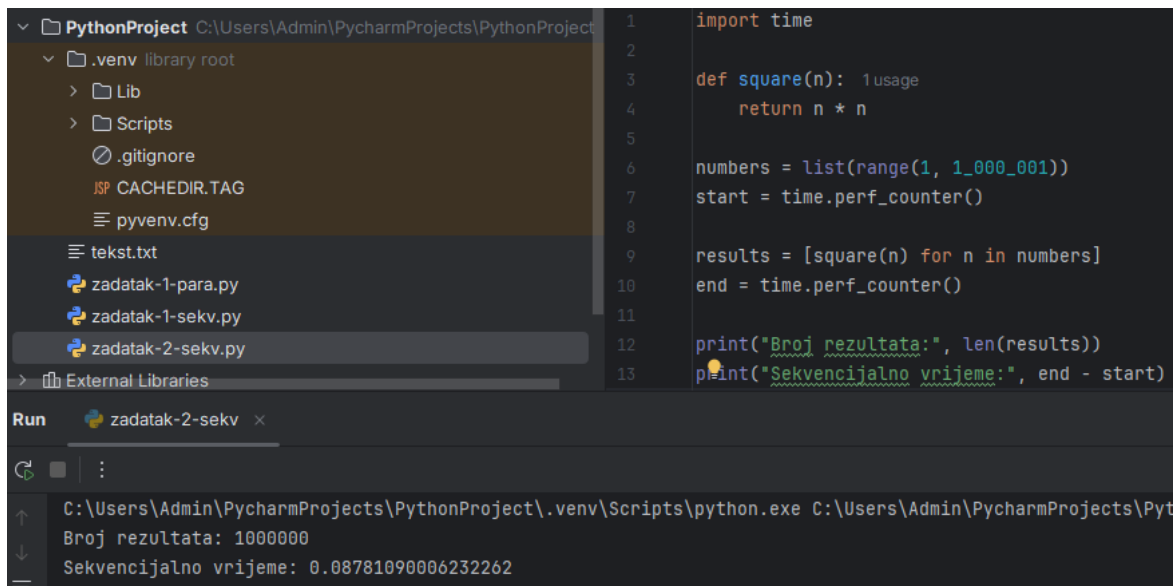
Slika 2 - Paralelna verzija sabiranja elemenata velike liste

Kod velikih lista paralelna verzija može donijeti ubrzanje jer se dijelovi liste obrađuju istovremeno. Ipak, Pythonova ugrađena funkcija `sum` je veoma optimizovana, pa se u nekim slučajevima može desiti da sekvencijalna verzija bude jednako dobra ili bolja. Paralelna verzija ima dodatni trošak zbog dijeljenja liste i prenosa podataka procesima.

3.2. Primjer 2: Izračunavanje kvadrata brojeva

Opis problema

Potrebno je izračunati kvadrate velikog broja cijelih brojeva.

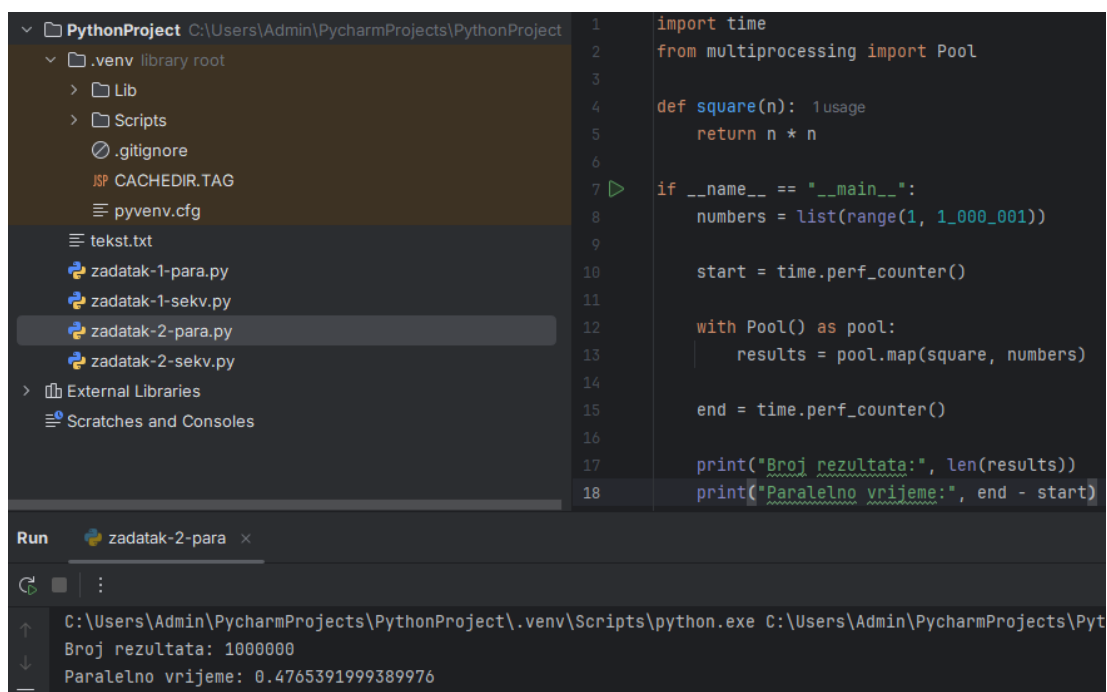


```
1 import time
2
3 def square(n):
4     return n * n
5
6 numbers = list(range(1, 1_000_001))
7 start = time.perf_counter()
8
9 results = [square(n) for n in numbers]
10 end = time.perf_counter()
11
12 print("Broj rezultata:", len(results))
13 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-2-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Pyt
Broj rezultata: 1000000
Sekvencijalno vrijeme: 0.08781090006232262

Slika 3 - Sekvencijalna verzija izračunavanja kvadrata brojeva



```
1 import time
2 from multiprocessing import Pool
3
4 def square(n):
5     return n * n
6
7 if __name__ == "__main__":
8     numbers = list(range(1, 1_000_001))
9
10    start = time.perf_counter()
11
12    with Pool() as pool:
13        results = pool.map(square, numbers)
14
15    end = time.perf_counter()
16
17    print("Broj rezultata:", len(results))
18    print("Paralelno vrijeme:", end - start)
```

Run zadatak-2-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Pyt
Broj rezultata: 1000000
Paralelno vrijeme: 0.4765391999389976

Slika 4 - Paralelna verzija izračunavanja kvadrata brojeva

Ovaj primjer pokazuje da paralelizam nije uvijek bolji. Operacija kvadriranja je veoma jednostavna, pa trošak slanja milion brojeva procesima može biti veći od koristi. Paralelna verzija bi bila korisnija kada bi funkcija square predstavljala mnogo složeniji proračun.

3.3. Primjer 3: Provjera prostih brojeva

Opis problema

Potrebno je provjeriti koliko brojeva iz zadatog intervala predstavlja proste brojeve. Provjera prostih brojeva je CPU-intenzivan zadatak jer zahtijeva veliki broj dijeljenja.

```
1  import time
2  import math
3
4  def is_prime(n): 1usage
5      if n < 2:
6          return False
7
8      for i in range(2, int(math.sqrt(n)) + 1):
9          if n % i == 0:
10             return False
11
12         return True
13
14     numbers = list(range(100_000, 120_000))
15
16     start = time.perf_counter()
17
18     results = [is_prime(n) for n in numbers]
19
20     end = time.perf_counter()
21
22     print("Broj prostih:", sum(results))
23     print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-3-sekv ×

↑ C:\Users\Admin\PycharmProjects\PythonProject\.ven
↓ Broj prostih: 1709
— Sekvencijalno vrijeme: 0.035729700000956655

Slika 5 - Sekvencijalna verzija provjere prostih brojeva

```
1 import time
2 import math
3 from multiprocessing import Pool
4
5 def is_prime(n): 1 usage
6     if n < 2:
7         return False
8
9     for i in range(2, int(math.sqrt(n)) + 1):
10         if n % i == 0:
11             return False
12
13     return True
14
15 if __name__ == "__main__":
16     numbers = list(range(100_000, 120_000))
17
18     start = time.perf_counter()
19
20     with Pool() as pool:
21         results = pool.map(is_prime, numbers)
22
23     end = time.perf_counter()
24
25     print("Broj prostih:", sum(results))
26     print("Paralelno vrijeme:", end - start)
```

Run zadatak-3-para x

C:\Users\Admin\PycharmProjects\PythonProject\.ve
Broj prostih: 1709
Paralelno vrijeme: 0.23648220009636134

Slika 6 - Paralelna verzija provjere prostih brojeva

Ovo je dobar primjer za multiprocessing, jer se svaki broj može provjeriti nezavisno. Paralelna verzija može ostvariti ubrzanje zato što više procesa istovremeno provjerava različite brojeve. Ubrzanje zavisi od veličine intervala i broja jezgara.

3.4. Primjer 4: Pretraga elementa u velikoj listi

Opis problema

Potrebno je provjeriti da li se određeni element nalazi u velikoj listi.

```
1 import time
2
3 numbers = list(range(1, 20_000_001))
4 target = 19_999_999
5
6 start = time.perf_counter()
7
8 found = target in numbers
9
10 end = time.perf_counter()
11
12 print("Pronađen:", found)
13 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-4-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.v
Pronađen: True
Sekvencijalno vrijeme: 0.21510939998552203

Slika 7 - Sekvencijalna verzija pretrage elemenata u velikoj listi

```
1 import time
2 from multiprocessing import Pool, cpu_count
3
4 def search_chunk(args): 1 usage
5     chunk, target = args
6     return target in chunk
7
8 if __name__ == "__main__":
9     numbers = list(range(1, 20_000_001))
10    target = 19_999_999
11
12    cores = cpu_count()
13    chunk_size = len(numbers) // cores
14
15    chunks = [
16        (numbers[i:i + chunk_size], target)
17        for i in range(0, len(numbers), chunk_size)
18    ]
19
20    start = time.perf_counter()
21
22    with Pool(cores) as pool:
23        results = pool.map(search_chunk, chunks)
24
25    found = any(results)
26
27    end = time.perf_counter()
28
29    print("Pronađen:", found)
30    print("Paralelno vrijeme:", end - start)
```

Run zadatak-4-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Sc
Pronađen: True
Paralelno vrijeme: 1.9011617000214756

Slika 8 - Paralelna verzija pretrage elemenata u velikoj listi

Paralelna pretraga može biti korisna kada je lista veoma velika i kada se element nalazi pri kraju ili ga nema. Ako se traženi element nalazi na početku liste, sekvencijalna verzija može biti brža jer će završiti pretragu skoro odmah. Ovaj primjer pokazuje da rezultat zavisi od podataka, a ne samo od algoritma.

3.5. Primjer 5: Sortiranje više nezavisnih lista

Opis problema

Potrebno je sortirati više nezavisnih lista. Svaka lista se može sortirati odvojeno, što ovaj zadatak čini pogodnim za paralelizaciju.

```
1 import time
2 import random
3
4 lists = [
5     [random.randint(a: 1, b: 1_000_000) for _ in range(100_000)]
6     for _ in range(8)
7 ]
8
9 start = time.perf_counter()
10
11 sorted_lists = [sorted(lst) for lst in lists]
12
13 end = time.perf_counter()
14
15 print("Broj sortiranih lista:", len(sorted_lists))
16 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-5-sekv x

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
↓ Broj sortiranih lista: 8
Skvencijalno vrijeme: 0.12836640002205968

Slika 9 - Sekvencijalna verzija sortiranja više nezavisnih lista

```
1 import time
2 import random
3 from multiprocessing import Pool
4
5 def sort_list(lst):
6     return sorted(lst)
7
8 if __name__ == "__main__":
9     lists = [
10         [random.randint(a=1, b=1_000_000) for _ in range(100_000)]
11         for _ in range(8)
12     ]
13
14     start = time.perf_counter()
15
16     with Pool() as pool:
17         sorted_lists = pool.map(sort_list, lists)
18
19     end = time.perf_counter()
20
21     print("Broj sortiranih lista:", len(sorted_lists))
22     print("Paralelno vrijeme:", end - start)
```

Run zadatak-6-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj sortiranih lista: 8
Paralelno vrijeme: 0.39585560001432896

Slika 10 - Paralelna verzija sortiranja više nezavisnih lista

Pošto su liste nezavisne, paralelna verzija može rasporediti sortiranje na više procesa. Ovo je primjer gdje paralelizam ima smisla. Međutim, ako su liste male, trošak kreiranja procesa može smanjiti ili potpuno poništiti korist.

3.6. Primjer 6: Brojanje riječi u tekstovima

Opis problema

Potrebno je prebrojati riječi u više velikih tekstova. Svaki tekst se obrađuje nezavisno.

```
1 import time
2
3 def count_words(text): 1usage
4     return len(text.split())
5
6 texts = [
7     "Python je programski jezik. " * 500_000,
8     "Paralelno programiranje koristi vise procesa. " * 500_000,
9     "Sekvencijalni program radi korak po korak. " * 500_000
10 ]
11
12 start = time.perf_counter()
13
14 results = [count_words(text) for text in texts]
15
16 end = time.perf_counter()
17
18 print("Ukupno riječi:", sum(results))
19 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-6-sekv x

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.
↓ Ukupno riječi: 7500000
— Sekvencijalno vrijeme: 0.5546034999424592

Slika 11 - Sekvencijalna verzija brojanja riječi u tekstovima

```
1 import time
2 from multiprocessing import Pool
3
4 def count_words(text):
5     return len(text.split())
6
7 if __name__ == "__main__":
8     texts = [
9         "Python je programski jezik. " * 500_000,
10        "Paralelno programiranje koristi više procesa. " * 500_000,
11        "Sekvencijalni program radi korak po korak. " * 500_000
12    ]
13
14    start = time.perf_counter()
15
16    with Pool() as pool:
17        results = pool.map(count_words, texts)
18
19    end = time.perf_counter()
20
21    print("Ukupno riječi:", sum(results))
22    print("Paralelno vrijeme:", end - start)
```

Run zadatak-6-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Ukupno riječi: 7500000
Paralelno vrijeme: 0.7895969999954104

Slika 12 - Paralelna verzija brojanja riječi u tekstovima

Brojanje riječi u velikim tekstovima može imati koristi od paralelizma, posebno ako postoji više tekstova koji se obrađuju odvojeno. Ako je tekst mali, paralelna verzija vjerovatno neće biti bolja.

3.7. Primjer 7: Rad sa više tekstualnih fajlova

Opis problema

Potrebno je pročitati više tekstualnih fajlova i izračunati ukupan broj karaktera. Pošto je čitanje fajlova I/O operacija, za paralelnu verziju koristi se ThreadPoolExecutor.

Sekvencijalna verzija

```
1 import time
2
3 file_names = ["file1.txt", "file2.txt", "file3.txt"]
4
5 def count_chars(file_name): 1 usage
6     try:
7         with open(file_name, "r", encoding="utf-8") as file:
8             return len(file.read())
9     except FileNotFoundError:
10        print(f"Fajl {file_name} nije pronađen.")
11        return 0
12
13 start = time.perf_counter()
14
15 results = [count_chars(name) for name in file_names]
16
17 end = time.perf_counter()
18
19 print("Ukupno karakteri:", sum(results))
20 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-7-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe

Fajl file1.txt nije pronađen.
Fajl file2.txt nije pronađen.
Fajl file3.txt nije pronađen.
Ukupno karakteri: 0
Sekvencijalno vrijeme: 0.00018360000103712082

Slika 13 - Sekvencijalna verzija rada sa više tekstualnih fajlova

```
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 file_names = ["file1.txt", "file2.txt", "file3.txt"]
5
6 def count_chars(file_name): 1 usage
7     try:
8         with open(file_name, "r", encoding="utf-8") as file:
9             return len(file.read())
10    except FileNotFoundError:
11        print(f"Fajl {file_name} nije pronađen.")
12        return 0
13
14 if __name__ == "__main__":
15     start = time.perf_counter()
16
17     with ThreadPoolExecutor() as executor:
18         results = list(executor.map(count_chars, *iterables: file_names))
19
20     end = time.perf_counter()
21
22     print("Ukupno karaktera:", sum(results))
23     print("Paralelno vrijeme:", end - start)
```

Run zadatak-7-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\
Fajl file1.txt nije pronađen.
Fajl file2.txt nije pronađen.
Fajl file3.txt nije pronađen.
Ukupno karaktera: 0
Paralelno vrijeme: 0.002126400009728968

Slika 14 - Paralelna verzija rada sa više tekstualnih fajlova

Kod rada sa fajlovima program često čeka disk. Zato niti mogu biti korisne, jer dok jedna nit čeka čitanje fajla, druga može raditi sa drugim fajlom. Ubrzanje zavisi od brzine diska, veličine fajlova i broja fajlova.

3.8. Primjer 8: Simulacija mrežnih zahtjeva

Opis problema

Potrebno je simulirati više mrežnih zahtjeva. Svaki zahtjev traje jednu sekundu. Ovaj primjer pokazuje veliku razliku između sekvencijalnog i paralelnog izvršavanja kod zadataka koji čekaju.

```
1 import time
2
3 def fake_request(n):
4     time.sleep(1)
5     return f"Zahtjev {n} završen"
6
7 start = time.perf_counter()
8
9 results = [fake_request(i) for i in range(5)]
10
11 end = time.perf_counter()
12
13 print(results)
14 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-8-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-8-sekv.py
['Zahtjev 0 završen', 'Zahtjev 1 završen', 'Zahtjev 2 završen', 'Zahtjev 3 završen', 'Zahtjev 4 završen']
Sekvencijalno vrijeme: 5.001243900042027

Slika 15 - Sekvencijalna verzija simulacije mrežnih zahtjeva

```
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 def fake_request(n):
5     time.sleep(1)
6     return f"Zahtjev {n} završen"
7
8 if __name__ == "__main__":
9     start = time.perf_counter()
10
11     with ThreadPoolExecutor(max_workers=5) as executor:
12         results = list(executor.map(fake_request, range(5)))
13
14     end = time.perf_counter()
15
16     print(results)
17     print("Paralelno vrijeme:", end - start)
```

Run zadatak-8-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-8-para.py
['Zahtjev 0 završen', 'Zahtjev 1 završen', 'Zahtjev 2 završen', 'Zahtjev 3 završen', 'Zahtjev 4 završen']
Paralelno vrijeme: 1.0016947000985965

Slika 16 - Paralelna verzija simulacije mrežnih zahtjeva

Sekvencijalna verzija traje približno pet sekundi jer se svaki zahtjev čeka posebno. Paralelna verzija traje približno jednu sekundu jer se svi zahtjevi pokreću istovremeno. Ovo je vrlo dobar primjer primjene niti.

3.9. Primjer 9: Obrada slike predstavljene matricom piksela

Opis problema

Slika se može predstaviti kao matrica piksela. U ovom primjeru svaki piksel se invertuje tako što se njegova vrijednost zamijeni vrijednošću 255 - piksel.

```
1 import time
2
3 image = [[i % 256 for i in range(1000)] for _ in range(1000)]
4
5 def process_pixel(pixel): 1 usage
6     return 255 - pixel
7
8 start = time.perf_counter()
9
10 processed = [
11     [process_pixel(pixel) for pixel in row]
12     for row in image
13 ]
14
15 end = time.perf_counter()
16
17 print("Broj redova:", len(processed))
18 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-9-sekv x

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python
↓ Broj redova: 1000
Sekvencijalno vrijeme: 0.06182840000838041

Slika 17 - Sekvencionalna verzija obrade slike predstavljene matricom piksela

```
1 import time
2 from multiprocessing import Pool
3
4 def process_row(row):
5     return [255 - pixel for pixel in row]
6
7 if __name__ == "__main__":
8     image = [[i % 256 for i in range(1000)] for _ in range(1000)]
9
10    start = time.perf_counter()
11
12    with Pool() as pool:
13        processed = pool.map(process_row, image)
14
15    end = time.perf_counter()
16
17    print("Broj redova:", len(processed))
18    print("Paralelno vrijeme:", end - start)
```

Run zadatak-9-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj redova: 1000
Paralelno vrijeme: 0.28695079998578876

Slika 18 - Paralelna verzija obrade slike predstavljene matricom piksela

Svaki red slike može se obrađivati nezavisno. Kod većih slika paralelizam može donijeti ubrzanje. Kod manjih slika dobitak može biti manji zbog troška prenosa redova slike procesima.

3.10. Primjer 10: Monte Carlo simulacija za približnu vrijednost broja pi

Opis problema

Monte Carlo metoda koristi slučajne tačke za približno određivanje vrijednosti broja pi. Pošto se svaka tačka generiše nezavisno, metoda je pogodna za paralelno izvršavanje.

```
1 import time
2 import random
3
4 def monte_carlo_pi(points): 1 usage
5     inside = 0
6
7     for _ in range(points):
8         x = random.random()
9         y = random.random()
10
11         if x * x + y * y <= 1:
12             inside += 1
13
14     return inside
15
16 points = 2_000_000
17
18 start = time.perf_counter()
19
20 inside = monte_carlo_pi(points)
21 pi = 4 * inside / points
22
23 end = time.perf_counter()
24
25 print("Približna vrijednost pi:", pi)
26 print("Sekvencijalno vrijeme:", end - start)
```

Run  zadatak-10-sekv x

  | :

↑ C:\Users\Admin\PycharmProjects\PythonProject\.v
Približna vrijednost pi: 3.13986
↓
Sekuencijalno vrijeme: 0.3264644999289885

Slika 19 - Sekvencijalna verzija Monte Carlo simulacije za približnu vrijednost broja pi

```
1 import time
2 import random
3 from multiprocessing import Pool, cpu_count
4
5 def monte_carlo_pi(points): 1 usage
6     inside = 0
7
8     for _ in range(points):
9         x = random.random()
10        y = random.random()
11
12        if x * x + y * y <= 1:
13            inside += 1
14
15    return inside
16
17 if __name__ == "__main__":
18     total_points = 2_000_000
19     cores = cpu_count()
20     points_per_process = total_points // cores
21
22     start = time.perf_counter()
23
24     with Pool(cores) as pool:
25         results = pool.map(monte_carlo_pi, [points_per_process] * cores)
26
27     inside = sum(results)
28     pi = 4 * inside / total_points
29
30     end = time.perf_counter()
31
32     print("Približna vrijednost pi:", pi)
33     print("Paralelno vrijeme:", end - start)
```

Run zadatak-10-para x

↑ Približna vrijednost pi: 3.14274
Paralelno vrijeme: 0.3290794000495225

Slika 20 - Paralelna verzija Monte Carlo simulacije za približnu vrijednost broja pi

Ovaj primjer je pogodan za paralelizam jer se simulacija lako dijeli na više nezavisnih dijelova. Svaki proces generiše dio tačaka, a na kraju se rezultati saberu. Paralelna verzija obično donosi dobro ubrzanje.

3.11. Primjer 11: Množenje matrica

Opis problema

Potrebno je pomnožiti dvije matrice. Svaki red rezultujuće matrice može se izračunati nezavisno.

```
1  import time
2
3  size = 200
4
5  A = [[1 for _ in range(size)] for _ in range(size)]
6  B = [[2 for _ in range(size)] for _ in range(size)]
7
8  start = time.perf_counter()
9
10 C = [[0 for _ in range(size)] for _ in range(size)]
11
12 for i in range(size):
13     for j in range(size):
14         for k in range(size):
15             C[i][j] += A[i][k] * B[k][j]
16
17 end = time.perf_counter()
18
19 print("Dimenzija rezultata:", len(C), "x", len(C[0]))
20 print("Sekvencijalno vrijeme:", end - start)
```

Run  zadatak-11-sekv ×

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Script
↓ Dimenzija rezultata: 200 x 200
Sekvencijalno vrijeme: 1.9248784999363124

Slika 21 - Sekvencijalna verzija množenja matrica

```
1 import time
2 from multiprocessing import Pool
3
4 def multiply_row(args): 1 usage
5     row, B = args
6     size = len(B)
7     result_row = []
8
9     for j in range(size):
10         total = 0
11         for k in range(size):
12             total += row[k] * B[k][j]
13         result_row.append(total)
14
15     return result_row
16
17 if __name__ == "__main__":
18     size = 200
19
20     A = [[1 for _ in range(size)] for _ in range(size)]
21     B = [[2 for _ in range(size)] for _ in range(size)]
22
23     tasks = [(row, B) for row in A]
24
25     start = time.perf_counter()
26
27     with Pool() as pool:
28         C = pool.map(multiply_row, tasks)
29
30     end = time.perf_counter()
31
32     print("Dimenzija rezultata:", len(C), "x", len(C[0]))
33     print("Paralelno vrijeme:", end - start)
```

Run zadatak-11-para x

Dimenzija rezultata: 200 x 200
Paralelno vrijeme: 0.3095727999461815

Slika 22 - Paralelna verzija množenja matrica

Množenje matrica je računski zahtjevan problem. Paralelna verzija može biti brža jer svaki proces računa određene redove rezultujuće matrice. Ipak, treba voditi računa da se matrica B šalje procesima, što može izazvati dodatni trošak kod velikih matrica.

3.12. Primjer 12: Izračunavanje faktoriijela

Opis problema

Potrebno je izračunati faktorijele za više velikih brojeva. Svaki faktorijel se može računati nezavisno.

```
1 import time
2 import math
3
4 numbers = [50_000, 51_000, 52_000, 53_000]
5
6 start = time.perf_counter()
7
8 results = [math.factorial(n) for n in numbers]
9
10 end = time.perf_counter()
11
12 print("Broj izračunatih faktoriijela:", len(results))
13 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-12-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts
Broj izračunatih faktoriijela: 4
Sekvencijalno vrijeme: 0.16724059998523444

Slika 23 - Sekvencijalna verzija izračunavanja faktoriijela

```
1 import time
2 import math
3 from multiprocessing import Pool
4
5 def factorial_number(n): 1 usage
6     return math.factorial(n)
7
8 ▶ if __name__ == "__main__":
9     numbers = [50_000, 51_000, 52_000, 53_000]
10
11     start = time.perf_counter()
12
13     with Pool() as pool:
14         results = pool.map(factorial_number, numbers)
15
16     end = time.perf_counter()
17
18     print("Broj izračunatih faktorijskih brojeva:", len(results))
19     print("Paralelno vrijeme:", end - start)
```

Run zadatak-12-para x

↻ | ⋮

↑
↓

```
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\
Broj izračunatih faktorijskih brojeva: 4
Paralelno vrijeme: 0.32969459996093065
```

Slika 24 - Paralelna verzija izračunavanja faktorijskih brojeva

Kod velikih brojeva računanje faktorijskih brojeva može biti zahtjevno. Paralelna verzija može koristiti više procesorskih jezgara. Ako su brojevi mali, sekvencijalna verzija može biti dovoljna i jednostavnija.

3.13. Primjer 13: Filtriranje velikog skupa podataka

Opis problema

Potrebno je izdvojiti brojeve koji su djeljivi i sa 5 i sa 7. Velika lista se može podijeliti na više dijelova.

```
1 import time
2
3 numbers = list(range(1, 10_000_001))
4
5 start = time.perf_counter()
6
7 filtered = [n for n in numbers if n % 5 == 0 and n % 7 == 0]
8
9 end = time.perf_counter()
10
11 print("Broj elemenata:", len(filtered))
12 print("Sekvencijalno vrijeme:", end - start)
```

Run  zadatak-13-sekv x

↻  | ⋮

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
↓ Broj elemenata: 285714
Sekuencijalno vrijeme: 0.42004110000561923

Slika 25 - Sekvencijalna verzija filtriranja velikog skupa podataka

```
1 import time
2 from multiprocessing import Pool, cpu_count
3
4 def filter_chunk(chunk): 1 usage
5     return [n for n in chunk if n % 5 == 0 and n % 7 == 0]
6
7 ▶ if __name__ == "__main__":
8     numbers = list(range(1, 10_000_001))
9     cores = cpu_count()
10    chunk_size = len(numbers) // cores
11
12    chunks = [
13        numbers[i:i + chunk_size]
14        for i in range(0, len(numbers), chunk_size)
15    ]
16
17    start = time.perf_counter()
18
19    with Pool(cores) as pool:
20        results = pool.map(filter_chunk, chunks)
21
22    filtered = []
23    for part in results:
24        filtered.extend(part)
25
26    end = time.perf_counter()
27
28    print("Broj elemenata:", len(filtered))
29    print("Paralelno vrijeme:", end - start)
```

Run zadatak-13-para x

↺ ⏏ ⋮

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\py
↓ Broj elemenata: 285714
— Paralelno vrijeme: 1.0654004999669269

Slika 26 - Paralelna verzija filtriranja velikog skupa podataka

Filtriranje velikog skupa podataka može biti pogodno za paralelizam. Svaki proces obrađuje svoj dio liste. Ipak, ako je uslov jednostavan, trošak prenosa podataka može biti značajan.

3.14. Primjer 14: Generisanje hash vrijednosti

Opis problema

Potrebno je izračunati SHA-256 hash vrijednosti za veliki broj stringova.

```
1  import time
2  import hashlib
3
4  def create_hash(text): 1 usage
5      return hashlib.sha256(text.encode()).hexdigest()
6
7  data = [f"tekst_{i}" for i in range(500_000)]
8
9  start = time.perf_counter()
10
11 hashes = [create_hash(text) for text in data]
12
13 end = time.perf_counter()
14
15 print("Broj hash vrijednosti:", len(hashes))
16 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-14-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts
Broj hash vrijednosti: 500000
Sekvencijalno vrijeme: 0.4811805000063032

Slika 27 - Sekvencijalna verzija za generisanje hash vrijednosti

```
1 import time
2 import hashlib
3 from multiprocessing import Pool
4
5 def create_hash(text):
6     return hashlib.sha256(text.encode()).hexdigest()
7
8 if __name__ == "__main__":
9     data = [f"tekst_{i}" for i in range(500_000)]
10
11     start = time.perf_counter()
12
13     with Pool() as pool:
14         hashes = pool.map(create_hash, data)
15
16     end = time.perf_counter()
17
18     print("Broj hash vrijednosti:", len(hashes))
19     print("Paralelno vrijeme:", end - start)
```

Run zadatak-14-para x

↑
↓

```
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scr
Broj hash vrijednosti: 500000
Paralelno vrijeme: 0.4986292999237776
```

Slika 28 - Paralelna verzija za generisanje hash vrijednosti

Hash funkcije mogu biti zahtjevne kada se izvršavaju nad velikim brojem elemenata. Paralelna verzija može biti brža, ali kod kratkih stringova trošak komunikacije između procesa može smanjiti dobitak.

3.15. Primjer 15: Simulacija više nezavisnih zadataka

Opis problema

Potrebno je izvršiti više nezavisnih zadataka koji traju određeno vrijeme. Ovaj primjer koristi sleep da simulira čekanje.

```
1 import time
2
3 def task(n): 1 usage
4     time.sleep(2)
5     return f"Zadatak {n} završen"
6
7 start = time.perf_counter()
8
9 results = [task(i) for i in range(4)]
10
11 end = time.perf_counter()
12
13 print(results)
14 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-15-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PythonProject\main.py
['Zadatak 0 završen', 'Zadatak 1 završen', 'Zadatak 2 završen', 'Zadatak 3 završen']
Sekvencijalno vrijeme: 8.000955399940722

Slika 29 - Sekvencijalna verzija simulacije više nezavisnih zadataka

```
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 def task(n): 1 usage
5     time.sleep(2)
6     return f"Zadatak {n} završen"
7
8 if __name__ == "__main__":
9     start = time.perf_counter()
10
11     with ThreadPoolExecutor(max_workers=4) as executor:
12         results = list(executor.map(task, *iterables: range(4)))
13
14     end = time.perf_counter()
15
16     print(results)
17     print("Paralelno vrijeme:", end - start)
```

Run zadatak-15-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PythonProject\main.py
['Zadatak 0 završen', 'Zadatak 1 završen', 'Zadatak 2 završen', 'Zadatak 3 završen']
Paralelno vrijeme: 2.001664699986577

Slika 30 - Paralelna verzija simulacije više nezavisnih zadataka

Sekvencijalna verzija traje približno osam sekundi, jer se četiri zadatka izvršavaju jedan za drugim. Paralelna verzija traje približno dvije sekunde, jer se zadaci izvršavaju istovremeno. Ovo je jedan od najjasnijih primjera koristi paralelizma kod zadataka koji čekaju.

ZAKLJUČAK

Sekvencijalno i paralelno programiranje predstavljaju dva različita pristupa organizaciji izvršavanja programa. Sekvencijalni pristup je jednostavniji, pregledniji i lakši za testiranje. Pogodan je za manje zadatke i za situacije u kojima se operacije moraju izvršavati tačno određenim redoslijedom.

Paralelno programiranje omogućava bolje iskorišćenje savremenih višejezgrenih procesora. Njegova najveća prednost je mogućnost smanjenja vremena izvršavanja kod velikih i nezavisnih zadataka. U Pythonu se za ovu svrhu mogu koristiti biblioteke `threading`, `multiprocessing` i `concurrent.futures`.

Na osnovu praktičnih primjera može se zaključiti da paralelizam najviše koristi kod CPU-intenzivnih problema kao što su provjera prostih brojeva, Monte Carlo simulacije i množenje matrica. Takođe je veoma koristan kod I/O zadataka, gdje program dosta vremena provodi čekajući, kao što su mrežni zahtjevi i rad sa fajlovima.

Ipak, paralelno programiranje uvodi dodatnu složenost. Potrebno je voditi računa o sinhronizaciji, komunikaciji između procesa, dijeljenju podataka i mogućim greškama kao što su race condition i deadlock. Zbog toga se paralelizam treba koristiti promišljeno, onda kada problem zaista ima osobine koje omogućavaju efikasno paralelno izvršavanje.

Opšti zaključak rada je da paralelno programiranje predstavlja važan pristup u savremenom softverskom inženjerstvu, ali da nije zamjena za dobro projektovanje algoritma. Najbolji rezultati postižu se kada se pravilno razumije priroda problema, kada se izabere odgovarajuća biblioteka i kada se performanse potvrde mjerenjem.

POPIS SLIKA

Slika 1 - Sekvencijalna verzija sabiranja elemenata velike liste.....	7
Slika 2 - Paralelna verzija sabiranja elemenata velike liste	7
Slika 3 - Sekvencijalna verzija izračunavanja kvadrata brojeva.....	8
Slika 4 - Paralelna verzija izračunavanja kvadrata brojeva	8
Slika 5 - Sekvencijalna verzija provjere prostih brojeva	9
Slika 6 - Paralelna verzija provjere prostih brojeva.....	10
Slika 7 - Sekvencijalna verzija pretrage elemenata u velikoj listi	11
Slika 8 - Paralelna verzija pretrage elemenata u velikoj listi.....	12
Slika 9 - Sekvencijalna verzija sortiranja više nezavisnih lista	13
Slika 10 - Paralelna verzija sortiranja više nezavisnih lista.....	14
Slika 11 - Sekvencijalna verzija brojanja riječi u tekstovima.....	15
Slika 12 - Paralelna verzija brojanja riječi u tekstovima	16
Slika 13 - Sekvencijalna verzija rada sa više tekstualnih fajlova	17
Slika 14 - Paralelna verzija rada sa više tekstualnih fajlova.....	18
Slika 15 - Sekvencijalna verzija simulacije mrežnih zahtjeva.....	19
Slika 16 - Paralelna verzija simulacije mrežnih zahtjeva	19
Slika 17 - Sekvencionalna verzija obrade slike predstavljene matricom piksela	20
Slika 18 - Paralelna verzija obrade slike predstavljene matricom piksela.....	21
Slika 19 - Sekvencijalna verzija Monte Carlo simulacije za približnu vrijednost broja pi.....	22
Slika 20 - Paralelna verzija Monte Carlo simulacije za približnu vrijednost broja pi	23
Slika 21 - Sekvencijalna verzija množenja matrica	24
Slika 22 - Paralelna verzija množenja matrica.....	25
Slika 23 - Sekvencijalna verzija izračunavanja faktoriijela	26
Slika 24 - Paralelna verzija izračunavanja faktoriijela.....	27
Slika 25 - Sekvencijalna verzija filtriranja velikog skupa podataka.....	28
Slika 26 - Paralelna verzija filtriranja velikog skupa podataka	29
Slika 27 - Sekvencijalna verzija za generisanje hash vrijednosti	30
Slika 28 - Paralelna verzija za generisanje hash vrijednosti	31
Slika 29 - Sekvencijalna verzija simulacije više nezavisnih zadataka.....	32
Slika 30 - Paralelna verzija simulacije više nezavisnih zadataka	32